Modified DH Parameter Tables — Unitree G1 Inspire-Hand DFQ

Convention: Craig MDH (1989) $T_{i-1\to i} = R_x(\alpha_{i-1})\,T_x(a_{i-1})\,R_z(\theta_i + \theta_{\text{off}})\,T_z(d_i)$

Base frame: `{L|R}_hand_base_link` (attached at wrist yaw + fixed URDF offset)

Finger	#	Joint	a	α	d	θ_{off}	Ax	q_{lo}	q_{hi}	Type
Left Hand										
Thumb	0	L_thumb_prox_yaw	-16.96	90	69.10	99.71	+1	-5.73	74.49	act
	1	L_thumb_prox_pitch	11.50	90	-8.00	38.10	-1	-5.73	34.38	act
	2	L_thumb_inter	56.00	0	0.80	-11.46	-1	-5.73	34.38	1.6x
	3	L_thumb_dist	22.65	0	1.20	63.36	-1	-5.73	34.38	2.4x
Index	0	L_index_prox	0.29	178	-27.48	94.32	-1	0	97.40	act
	1	L_index_inter	32.13	0	1.00	85.68	-1	0	97.40	1.0x
Middle	0	L_middle_prox	137.10	180	-12.95	4.44	-1	0	97.40	act
	1	L_middle_inter	32.13	0	1.00	176.40	-1	0	97.40	1.0x
Ring	0	L_ring_prox	-0.29	176.99	-0.89	-85.68	-1	0	97.40	act
	1	L_ring_inter	32.13	0	1.00	-94.32	-1	0	97.40	1.0x
Pinky	0	L_pinky_prox	-0.29	174	11.16	-85.68	-1	0	97.40	act
	1	L_pinky_inter	32.13	0	1.00	-94.32	-1	0	97.40	1.0x
Right Hand										
Thumb	0	R_thumb_prox_yaw	16.96	90	-69.10	106.21	-1	-5.73	74.49	act
	1	R_thumb_prox_pitch	-11.50	90	8.00	-141.90	+1	-5.73	34.38	act
	2	R_thumb_inter	56.00	0	-0.80	-11.46	+1	-5.73	34.38	1.6x
	3	R_thumb_dist	22.65	0	-1.20	-116.64	+1	-5.73	34.38	2.4x
Index	0	R_index_prox	-0.29	178	27.49	-85.33	+1	0	97.40	act
	1	R_index_inter	32.13	0	-1.00	-94.67	+1	0	97.40	1.0x
Middle	0	R_middle_prox	0.29	180	12.95	94.32	+1	0	97.40	act
	1	R_middle_inter	32.13	0	-1.00	85.68	+1	0	97.40	1.0x
Ring	0	R_ring_prox	0.29	176.99	0.89	94.32	+1	0	97.40	act
	1	R_ring_inter	32.13	0	-1.00	85.68	+1	0	97.40	1.0x
Pinky	0	R_pinky_prox	0.29	174	-11.17	94.32	+1	0	97.40	act
	1	R_pinky_inter	32.13	0	-1.00	85.68	+1	0	97.40	1.0x

Notes.

- The MDH convention used is Craig (1989): the α and a parameters describe the *previous* link, so frame i is built from the i-th joint.
- $\alpha \approx 90°$ on both thumb joints indicates the thumb rotates *out of the finger plane* (abduction/adduction + flexion).
- $\alpha \approx 180°$ on non-thumb proximal joints reflects the URDF convention that all proximal joints rotate in a direction antiparallel to the hand-base z-axis.
- The $a = 137.10$ mm entry for L/R Middle proximal encodes the **hand-base to proximal-joint** Y-offset; all other non-thumb proximal a values (± 0.29 mm) are effectively zero (numerical rounding of the common-perpendicular algorithm for nearly-parallel axes).
- Mimic joints are not independently actuated: their joint angle is set by the URDF `<mimic>` tag to a fixed multiple of the parent proximal joint. The scale factors are 1.0× (index/middle/ring/pinky intermediate), 1.6× (thumb intermediate), and 2.4× (thumb distal).
- All MDH rows satisfy self-consistency error $< 4 \times 10^{-9}$ rad/m (float64 rounding). The URDF-chain FK (`fk_dh`) matches `fk_fingers` to < 0.001 mm over 200 random configurations.

article amsmath amssymb

1 Kinematic Implementation Overview

- `fk_fingers.py` — Ground truth implementation. It hard-codes URDF joint origins (`xyz`, `rpy`), joint axes, and mimic multipliers for the full chain (pelvis → waist → arm → wrist → hand base → fingers). Forward kinematics (FK) is computed via direct composition:

$$T \leftarrow T \cdot \text{Fixed}(xyz,\, rpy) \cdot \text{Rotate}(\text{axis},\, q)$$

  This produces the "verified to < 0.001 mm" reference results.

- `dh_table.py` — Automatically derives a Modified Denavit–Hartenberg (MDH) table from the same URDF data. This serves as a structured representation and sanity check, but is not used for FK computation.

- `dh_kinematics.py` — Controller-facing wrapper. Although it exposes `fk_dh()`, it does not use the MDH rows for computation; instead, it re-walks the URDF chain directly. The MDH table is retained only for inspection and printing.

Thus, the URDF chain is the single source of truth.

2 MDH Convention

Craig's Modified DH convention (1989) is used:

$$T_{i-1\to i} = R_x(\alpha_{i-1}) \cdot T_x(a_{i-1}) \cdot R_z(\theta_i + \theta_{\text{off}}) \cdot T_z(d_i)$$

For joint i:

- a: distance along x_{i-1} between consecutive z-axes
- α: angle from z_{i-1} to z_i about x_{i-1}
- d: translation along z_i
- θ_{off}: angle between x_{i-1} and x_i about z_i at $q = 0$

3 MDH Parameter Extraction

3.1 Step 1: URDF Chain Traversal

At $q = 0$, the chain is traversed once. For each joint:

- z_k: joint axis expressed in the hand-base frame
- o_k: joint origin position in the hand-base frame

3.2 Step 2: Canonical Frame Construction

The DH x-axis is defined as the common perpendicular between consecutive z-axes:

$$x = \begin{cases} \dfrac{z_a \times z_b}{\|z_a \times z_b\|}, & \text{if not parallel} \\ \text{projected origin difference}, & \text{if parallel} \end{cases}$$

Each frame is completed as:

$$y = z \times x$$

3.3 Step 3: MDH Scalars

For consecutive frames:

$$\alpha = \text{atan2}\big((z_{i-1} \times z_i) \cdot x_{i-1},\ z_{i-1} \cdot z_i\big) \tag{1}$$

$$a = (o_i - o_{i-1}) \cdot x_{i-1} \tag{2}$$

$$d = (o_i - o_{i-1}) \cdot z_i \tag{3}$$

$$\theta_{\text{off}} = \text{atan2}\big((x_{i-1} \times x_i) \cdot z_i,\ x_{i-1} \cdot x_i\big) \tag{4}$$

3.4 Step 4: Mimic Mapping

Mimic joints are tagged using:

- Reference joint name
- Mimic scaling factors (1.0, 1.6, 2.4)

4 MDH Limitation and 137 mm Offset

The proximal joints have an offset of approximately 137 mm along the $-Y$ direction:

$$[0.00028533,\ -0.1371,\ 0.01295]$$

MDH can only represent offsets along X (via a) and Z (via d), not Y. Therefore:

- A pure MDH chain would introduce a systematic $\sim$137 mm error.
- The implementation avoids this by not using MDH rows for FK.

Instead, the call chain is:

`fk_dh()` $\rightarrow$ `_fk_urdf_chain()`

ensuring exact agreement with the URDF-based FK.

5 Interpretation of Table Columns

- a, α, d, θ_{off}: MDH parameters (displayed in mm and degrees)
- Axis sign: sign(axis$_z$)
- Joint limits: extracted from `HAND12_LIMITS`
- Type: actuated (`act`) or mimic (`1.6x` / `2.4x`)

The 137.10 mm value in a is a projection artifact from the canonical frame construction, not real geometry.

6 Usage in Controller

`DHFingertipKinematics.fk_tips()`:

- Computes fingertip pose per finger using `fk_dh()`
- Left-multiplies by the live hand-base transform T_{hb}
- Uses URDF-consistent FK throughout

The Jacobian is computed via finite differences:

$$J \approx \frac{\partial x}{\partial q}$$

7 Design Rationale

Using a single FK implementation avoids inconsistencies between:

- URDF-based FK
- Featherstone-tree FK (`ph_system.py`)

This ensures stable impedance control:

$$\tau = J_c^T \left(K_t(x_d - x) + \cdots \right)$$

Consistency in x is critical for numerical stability of the impedance term.

8 inverse Kinematics

9 Overview

This document is a LaTeX conversion of the Python module `ik_fingers_analytical.py`. It implements analytical inverse kinematics for the Inspire-Hand DFQ proximal finger joints, replacing a damped least-squares iterative loop with closed-form or near-closed-form solvers.

10 Mathematical Summary

For each non-thumb finger, the fingertip position in the proximal-joint body frame follows

$$p_{\text{tip}}(q) = p_{1,\text{fixed}} + R_{1,\text{fixed}} R_z(-q) p_{\text{link}}.$$

Given a target fingertip position, the joint angle is recovered through projection in the proximal frame using

$$v = R_{1,\text{fixed}}^T (p_{\text{des}} - p_{1,\text{fixed}}),$$

$$q = -\operatorname{atan2}(p_x v_y - p_y v_x,\ p_x v_x + p_y v_y).$$

The thumb uses a two-stage solution: yaw from azimuth, then pitch from a reduced 1-D nonlinear solve with Newton refinement.

11 Python Source

```
"""
ik_fingers_analytical.py
========================
Analytical (closed-form) IK for the Inspire-Hand DFQ proximal finger joints.
Replaces the DLS loop in ik_fingers.py for the HAND12 joints.


Theory
------
Each non-thumb finger (index/middle/ring/pinky) has ONE actuated DOF (proximal
joint, axis =[0,0,-1]) with the intermediate joint mimicking it 1:1. Because
the intermediate offset is a fixed vector that rotates together with the proximal
```

```
joint, the fingertip traces a CIRCLE in the proximal-joint body frame.


  p_tip(q) = p1_fixed + R1_fixed @ Rz(-q) @ p_link_vec


where:
  p1_fixed  = proximal joint origin in hand_base frame  (from URDF)
  R1_fixed  = fixed RPY rotation at the proximal joint   (from URDF)
  p_link_vec= offset to the intermediate joint (= fingertip) in the
              proximal child frame  (from URDF)
  q         = proximal joint angle (positive = curl inward)


Given a desired fingertip p_des, find q analytically:


  v = R1_fixed.T @ (p_des - p1_fixed)          # target in proximal
     body frame
  q = -atan2(px*vy - py*vx,  px*vx + py*vy)    # angle from p_link_xy
     to v_xy


where (px,py) = p_link_vec[:2].  This is exact when ||v_xy|| =
    ||p_link_xy|| and
we project v onto the reachable circle otherwise.


Thumb (2 DOF: yaw + pitch)
  Step 1 --- Yaw: solve from target's azimuth in the yaw-joint body
     frame.
  Step 2 --- Pitch: given fixed yaw, express target in pitch-joint
     frame and
             solve the 3-link effective chain (pitch + 1.6xpitch +
                2.4xpitch)
             using a fast 1-D Newton step (<=4 iterations, always
                converges).


Input/Output conventions
------------------------
theta29 (29,) = [waist(3), left_arm(7), right_arm(7), left_hand6(6),
    right_hand6(6)]
  --- same as fk_fingers.py


q_hand6 (6,) per hand = [thumb_yaw, thumb_pitch, index, middle, ring,
    pinky]
  --- this matches HAND12_SIM_INDICES order within each hand


Isaac Lab HAND12 sim indices (from robot_indices.py):
  Left : [29, 30, 33, 35, 37, 39]
  Right: [41, 42, 45, 47, 49, 51]
"""


from __future__ import annotations
```

```python


from dataclasses import dataclass
from typing import Dict, Optional, Tuple


import numpy as np


from .fk_fingers import (
    _Rx, _Ry, _Rz, _T, rpy2R_urdf,
    _fk_to_wrist_yaw_left, _fk_to_wrist_yaw_right,
    L_BASE_XYZ, L_BASE_RPY,
    R_BASE_XYZ, R_BASE_RPY,
    L_FINGER_CHAINS, R_FINGER_CHAINS,
    fk_fingers,
)
from .robot_indices import HAND12_LIMITS_LO, HAND12_LIMITS_HI


ArrayLike = np.ndarray


# Precomputed non-thumb chain data: (origin_xyz, origin_rpy, link_vec, axis_sign)
# link_vec = xyz of the intermediate (mimic) joint in the proximal child frame.
_L_NON_THUMB: dict[str, tuple] = {
    'index':  (L_FINGER_CHAINS['index'][0][0],
       L_FINGER_CHAINS['index'][0][1],
       L_FINGER_CHAINS['index'][1][0],  -1),
    'middle': (L_FINGER_CHAINS['middle'][0][0],
       L_FINGER_CHAINS['middle'][0][1],
       L_FINGER_CHAINS['middle'][1][0], -1),
    'ring':   (L_FINGER_CHAINS['ring'][0][0],
       L_FINGER_CHAINS['ring'][0][1],   L_FINGER_CHAINS['ring'][1][0],
         -1),
    'pinky':  (L_FINGER_CHAINS['pinky'][0][0],
       L_FINGER_CHAINS['pinky'][0][1],
       L_FINGER_CHAINS['pinky'][1][0],  -1),
}
_R_NON_THUMB: dict[str, tuple] = {
    'index':  (R_FINGER_CHAINS['index'][0][0],
       R_FINGER_CHAINS['index'][0][1],
       R_FINGER_CHAINS['index'][1][0],  +1),
    'middle': (R_FINGER_CHAINS['middle'][0][0],
       R_FINGER_CHAINS['middle'][0][1],
       R_FINGER_CHAINS['middle'][1][0], +1),
    'ring':   (R_FINGER_CHAINS['ring'][0][0],
       R_FINGER_CHAINS['ring'][0][1],   R_FINGER_CHAINS['ring'][1][0],
         +1),
    'pinky':  (R_FINGER_CHAINS['pinky'][0][0],
       R_FINGER_CHAINS['pinky'][0][1],
       R_FINGER_CHAINS['pinky'][1][0],  +1),
}


_L_THUMB_CHAIN = L_FINGER_CHAINS['thumb']
```

```python
_R_THUMB_CHAIN = R_FINGER_CHAINS['thumb']



def _finger_ik_1dof(
    p_des_in_handbase: np.ndarray,
    origin_xyz: list,
    origin_rpy: list,
    link_vec: list,
    axis_sign: int,
    q_lo: float,
    q_hi: float,
) -> float:
    """
    Analytical IK for a 1-DOF finger (index/middle/ring/pinky).


    The proximal joint rotates about its local Z with sign `axis_sign`.
    The fingertip = intermediate joint origin = proximal_child_frame
    displaced by `link_vec` and then rotated by the same joint angle
       (mimic 1.0).


    Because the intermediate joint is a FIXED frame step (no extra
       rotation),
    the fingertip in the PROXIMAL BODY FRAME is just:
       p_tip_local = Rz(axis_sign * q) @ link_vec


    and in hand_base frame:
       p_tip = p1 + R1_fixed @ p_tip_local


    Inversion:
       v = R1_fixed.T @ (p_des - p1)         (target in proximal body
          frame)
       q = -axis_sign * atan2( px*vy - py*vx,
                               px*vx + py*vy )


    where (px,py) = link_vec[:2].  Project v onto the reachable XY
       circle if
    the target is out of reach.
    """
    p1 = np.asarray(origin_xyz, dtype=float)
    R1 = rpy2R_urdf(origin_rpy)
    lv = np.asarray(link_vec, dtype=float)


    v = R1.T @ (p_des_in_handbase - p1)


    px, py = lv[0], lv[1]
    vx, vy = v[0], v[1]


    r_v  = np.hypot(vx, vy)
    r_lv = np.hypot(px, py)
```

```
156
157
158     if r_v < 1e-9 or r_lv < 1e-9:
159         return 0.0
160
161
162     # project v onto the reachable circle (radius = r_lv)
163     vx = vx * (r_lv / r_v)
164     vy = vy * (r_lv / r_v)
165
166
167     cross = px * vy - py * vx
168     dot   = px * vx + py * vy
169
170
171     angle_to_v = np.arctan2(cross, dot)
172     q = angle_to_v / axis_sign
173
174
175     return float(np.clip(q, q_lo, q_hi))
176
177
178
179 def _thumb_ik_2dof(
180     p_des_in_handbase: np.ndarray,
181     thumb_chain: list,
182     axis_sign_yaw: int,
183     axis_sign_pitch: int,
184     yaw_lo: float,
185     yaw_hi: float,
186     pitch_lo: float,
187     pitch_hi: float,
188 ) -> tuple[float, float]:
189     """
190     Analytical thumb IK (yaw + pitch).
191
192
193     Step 1 --- Yaw:
194       Transform target to yaw-joint body frame, solve azimuth angle.
195
196
197     Step 2 --- Pitch:
198       With yaw fixed, transform target to pitch-joint body frame.
199       The effective tip distance from the pitch-joint origin is:
200         r(p) = ||[l1*cos(p) + l2*cos(2.6p),
201                   l1*sin(p) + l2*sin(2.6p)]||
202
203
204       Solve via law-of-cosines approximation then 1-step Newton
            refinement.
205     """
206     yaw_xyz,   yaw_rpy,   yaw_ax,   _ = thumb_chain[0]
207     pitch_xyz, pitch_rpy, pitch_ax, _ = thumb_chain[1]
208     inter_xyz, inter_rpy, inter_ax, _ = thumb_chain[2]
209     dist_xyz,  dist_rpy,  dist_ax,  _ = thumb_chain[3]
210
211
212     p_yaw = np.asarray(yaw_xyz, dtype=float)
```

```
    R_yaw = rpy2R_urdf(yaw_rpy)


    p_pitch_local = np.asarray(pitch_xyz, dtype=float)
    R_pitch = rpy2R_urdf(pitch_rpy)


    l1 = float(np.linalg.norm(inter_xyz))
    l2 = float(np.linalg.norm(dist_xyz))


    m1, m2 = 1.0, 2.6
    dm = m2 - m1


    v_yaw_body = R_yaw.T @ (p_des_in_handbase - p_yaw)


    if np.hypot(v_yaw_body[0], v_yaw_body[1]) < 1e-6:
        q_yaw = 0.0
    else:
        ref_angle = np.arctan2(p_pitch_local[1], p_pitch_local[0])
        tgt_angle = np.arctan2(v_yaw_body[1],    v_yaw_body[0])
        angle_diff = tgt_angle - ref_angle
        q_yaw = angle_diff / axis_sign_yaw


    q_yaw = float(np.clip(q_yaw, yaw_lo, yaw_hi))


    T_yaw_joint = _T(R_yaw, p_yaw)
    Rz_yaw      = _Rz(-q_yaw)
    T_yaw_child = T_yaw_joint @ _T(Rz_yaw, [0.0, 0.0, 0.0])
    T_pitch_joint = T_yaw_child @ _T(R_pitch, p_pitch_local)


    p_pitch_hb    = T_pitch_joint[:3, 3]
    R_pitch_world = T_pitch_joint[:3, :3]
    v_pitch_body  = R_pitch_world.T @ (p_des_in_handbase - p_pitch_hb)


    r_des = float(np.hypot(v_pitch_body[0], v_pitch_body[1]))


    r_des_sq  = r_des ** 2
    cos_arg   = (r_des_sq - l1**2 - l2**2) / (2.0 * l1 * l2 + 1e-12)
    cos_arg   = float(np.clip(cos_arg, -1.0, 1.0))
    p_init    = np.arccos(cos_arg) / dm


    for _ in range(4):
        r_p     = np.sqrt(l1**2 + l2**2 + 2*l1*l2*np.cos(dm*p_init)) +
           1e-30
        dr_dp   = -2*l1*l2*dm*np.sin(dm*p_init) / (2*r_p + 1e-30)
        residual = r_p - r_des
        if abs(dr_dp) < 1e-10:
            break
        p_init -= residual / dr_dp
```

```
270
271
272     q_pitch = float(np.clip(p_init, pitch_lo, pitch_hi))
273     return q_yaw, q_pitch
274
275
276 @dataclass
277 class FingerIKResult:
278     q_sol:            np.ndarray
279     q_hand6_L:        np.ndarray
280     q_hand6_R:        np.ndarray
281     roundtrip_errors: dict
282
283
284 # Remaining helper and public functions omitted here for brevity in
      the LaTeX
285 # listing preview. Insert the rest of the original source if a full
      verbatim
286 # archival listing is required.
```

12 Notes

- The non-thumb fingers use exact geometric inversion in the proximal frame, with radial projection for unreachable targets.
- The thumb is handled separately because it has coupled yaw and pitch behavior.
- Joint limits are enforced with clipping after each analytical solve.
- The module includes optional FK validation and a direct curl-to-joint mapping utility.